

Session 7 Speaker Script — COMPLETE WITH FULL TALKING POINTS

Evaluation & Benchmarking

Presenter: Dinesh Wadhvani | **Session:** 7 of 12 | **Duration:** ~60 minutes

Purpose: Systematically measure AI system performance

Slide	Title	Time
1	Title Slide	~1 min
2	Today's Journey	~2 min
3	Why Evaluation Matters	~5 min
4	Evaluation Framework	~8 min
5	Key Metrics for AI	~8 min
6	Human Evaluation	~8 min
7	Automatic Metrics	~8 min
8	A/B Testing & Significance	~8 min
9	Building Benchmark Datasets	~5 min
10	Real-World Evaluation	~8 min
11	Continuous Monitoring	~5 min
12	Lab Overview	~3 min
13	Key Takeaways	~5 min
14	Session 8 Preview	~2 min
—	Lab Exercise	~12 min

01. TITLE SLIDE

~1 minute

WHAT'S ON SCREEN

Session 7 label (blue) • Title: 'Evaluation & Benchmarking' • Subtitle: 'Measuring What Matters' • Session 7 of 12

SPEAKING SCRIPT

Welcome to Session 7. We've now covered five critical areas: how LLMs work (Session 2), how to prompt them effectively (Session 3), how to structure their outputs (Session 4), how to ground them in real documents (Session 5), and how to specialize them for domains (Session 6).

Now comes the essential question that ties everything together: How do we know if any of it is actually working?

This is evaluation. And it's arguably more important than building the system itself. Because here's the hard truth: you can build a beautiful AI system. It can feel sophisticated. It can sound authoritative. But if you haven't measured whether it's actually better than the alternative, you don't know if it's worth the investment.

Evaluation is how you go from 'I built something' to 'I built something that works.' It's the difference between a project and a product.

In this session, you'll learn how to systematically measure AI performance. You'll understand metrics—both automatic and human. You'll learn statistical testing so you know when improvements are real vs random chance. You'll see how real companies evaluate before shipping to production.

By the end, you'll be able to take any AI system you've built and rigorously prove whether it's better than baselines. You'll have benchmarks. You'll have data. You'll have confidence.

This is where theory meets practice. This is where engineering begins.

DELIVERY TIPS

This is a pivotal moment—students often skip evaluation and just deploy. Emphasize it's essential. • 'How do you know it works?' is the key question. • Real companies spend enormous time here. • Set the tone: rigorous, systematic, data-driven.

02. TODAY'S JOURNEY

~2 minutes

WHAT'S ON SCREEN

14-item agenda • Items: Why evaluation, Framework, Metrics, Human eval, Auto metrics, A/B testing, Benchmarks, Real examples, Monitoring, Lab

SPEAKING SCRIPT

Quick roadmap. We start with five minutes on why evaluation matters. Not just 'why should you do it' but what happens when you don't. Real cases where companies shipped bad AI.

Then eight minutes on the evaluation framework. How do you structure thinking about what to measure?

Eight minutes on key metrics for AI systems. There are many. Which ones matter for your problem?

Eight minutes on human evaluation. Sometimes the best judge is a person. How do you structure that?

Eight minutes on automatic metrics. Scalable, objective, repeatable. How do you implement them?

Eight minutes on A/B testing and statistical significance. This is critical: how do you know an improvement is real and not random chance?

Five minutes on building benchmark datasets. What should you test on?

Eight minutes on real-world evaluation examples. How do companies do this?

Five minutes on continuous monitoring. Evaluation doesn't end at deployment. It continues.

Then lab—12 minutes. You'll evaluate your system against baselines using real metrics.

Then takeaways and Session 8 preview. If time gets tight, we compress the metrics section. We protect A/B testing, benchmarks, and lab because those are most practical.

DELIVERY TIPS

Set expectation: evaluation is deep, not shallow. • Multiple approaches (human, automatic, A/B testing) show this isn't one thing. • Monitoring at the end shows evaluation is ongoing. • Lab is where it gets real.

03. WHY EVALUATION MATTERS

~5 minutes

WHAT'S ON SCREEN

Consequences visual • Companies that didn't evaluate before shipping • Real examples of failures • Business and user impact

SPEAKING SCRIPT

Let me start with consequences. Why does evaluation matter? Because the alternative is disaster.

REAL EXAMPLE 1: AMAZON HIRING AI

What happened: Amazon built an AI system to review resumes and rank candidates. Seemed good. Never properly evaluated it against real hiring outcomes.

Problem discovered: The AI was biased against women. It had learned patterns from historical hiring data that favored men.

Why it happened: They didn't evaluate for bias. They didn't compare outcomes across demographics. They shipped without proper testing.

Business impact: Negative press coverage. Loss of trust. Had to shut down the system. Tens of millions of dollars invested, unusable.

Lesson: Evaluation would have caught this before deployment. A simple statistical test comparing acceptance rates by gender would have revealed the bias immediately.

REAL EXAMPLE 2: HEALTHCARE AI THAT SOUNDS GOOD BUT ISN'T

What happened: A company developed AI to diagnose rare diseases. In their internal tests, it seemed 95% accurate. They were excited to deploy it in hospitals.

Problem discovered: When tested on real patients from different hospitals, accuracy dropped to 72%.

Why it happened: They evaluated on their own data. But their data wasn't representative of all hospitals. Different equipment, different patient populations, different data quality. They didn't test on independent, diverse data.

Business impact: System deployed and gave incorrect diagnoses. One patient received wrong treatment. Could have been catastrophic. System was pulled, and trust in their company's AI damaged.

Lesson: Evaluation must include diverse test data. Your training data and test data must represent real-world conditions.

REAL EXAMPLE 3: CUSTOMER SERVICE BOT THAT FRUSTRATED CUSTOMERS

What happened: Company deployed an AI customer service bot. By all metrics they measured, it was working. Average response time down. Cost per support ticket down.

Problem discovered: Customer satisfaction actually went down. Customers were frustrated because the bot couldn't handle nuanced issues and was repetitive.

Why it happened: They measured cost and speed. They didn't measure what customers actually cared about: being heard, getting their problem solved, feeling valued.

Business impact: Customers left for competitors. Revenue impact. Had to partially roll back the AI and add human escalation paths.

Lesson: Measure what matters. Not what's easy to measure. If you measure the wrong thing, you optimize for the wrong thing.

WHAT THESE HAVE IN COMMON:

1. Teams thought their system was working without rigorous evidence
2. They didn't evaluate comprehensively (bias, out-of-distribution data, user satisfaction)
3. They didn't test on real-world conditions before deployment
4. The cost of failure was high (money, reputation, user harm)

THE EVALUATION MINDSET:

Assume your system is broken until proven otherwise. Not cynicism. Just engineering. You want evidence before you ship.

Questions to ask about ANY AI system you build:

- What could go wrong?
- How would I detect it?
- What metrics measure success?
- Who should evaluate it? (Humans? Automatic tests? Both?)
- What's the baseline I'm comparing against?
- Am I measuring real-world performance or just lab performance?
- Have I tested on diverse data?
- Could there be hidden biases?

REAL COMPANIES THAT GET THIS RIGHT:

Google: Before shipping any search algorithm change, they run A/B tests on millions of queries. They measure: click-through rate, query reformulations, user satisfaction, search intent match. Changes must beat baselines on MULTIPLE metrics, not just one.

OpenAI: Before releasing new models, they have extensive evaluation pipelines. They test on benchmarks (academic datasets), adversarial examples (things designed to trick the model), real-world use cases. They measure accuracy, bias, safety, toxicity, helpfulness. Only after rigorous evaluation do they release.

Medical device companies: Before FDA approval, medical AI must go through clinical trials. Real patients, diverse populations, statistical testing. Because the cost of failure is life and death.

THE BUSINESS CASE:

Investing in evaluation costs money upfront. But it saves money in the long run.

- Cost of fixing problems before deployment: Maybe \$100K
- Cost of fixing problems after deployment: Often millions (fixing the system, damage control, lost customers, reputation)

Companies that evaluate rigorously save money and build trust.

DELIVERY TIPS

Real examples with names (Amazon, medical AI, customer service) make this concrete. • Show specific consequences: bias, wrong diagnoses, lost customers. • Emphasize: evaluation is about confidence, not just checking boxes. • The comparison to medical devices shows seriousness. • Business case: evaluation is investment in risk management.

04. EVALUATION FRAMEWORK

~8 minutes

WHAT'S ON SCREEN

Evaluation framework diagram • Goal → Metrics → Data → Evaluation → Results → Decisions • Each component explained • Real example

SPEAKING SCRIPT

Here's how to think about evaluation systematically. It's not just 'run some tests.' It's a framework with clear components.

COMPONENT 1: DEFINE YOUR GOAL

What are you trying to achieve? This seems obvious but many teams skip it.

Example goals:

- 'This resume evaluation AI should identify strong candidates 90% of the time'
- 'This customer service bot should resolve 80% of issues on first interaction'
- 'This medical diagnostic AI should match specialist diagnosis 95% of the time'
- 'This content moderation AI should catch 99% of policy violations with less than 5% false positives'

Each goal is specific. Measurable. Achievable. Time-bound.

Bad goals: 'The system should work well.' 'The AI should be accurate.' Too vague.

Your goal becomes the north star for evaluation. Everything else flows from it.

FOR YOUR HIRING DOMAIN:

What's your goal? Example: 'This hiring recommendation AI should correctly identify top-quartile candidates (those who succeed in the role) 85% of the time, while rejecting bottom-quartile candidates (those who fail) 90% of the time.'

That's specific and measurable.

COMPONENT 2: CHOOSE YOUR METRICS

Now that you know your goal, what metrics measure whether you've achieved it?

Example: If your goal is 'identify top candidates 85% of the time,' your metric might be 'Recall on top-quartile candidates' (out of all top-quartile candidates in the real world, how many did the AI identify?).

But you also care about not falsely promoting bad candidates. So another metric: 'Precision on top-quartile' (of all candidates the AI promoted, what % are actually top-quartile?).

You typically need multiple metrics because one metric alone doesn't tell the full story.

COMPONENT 3: GATHER DATA FOR EVALUATION

You need test data. This data must:

- Be representative (similar to real-world conditions)
- Be diverse (cover different scenarios)
- Be labeled correctly (ground truth is accurate)
- Be large enough (statistically meaningful)
- Be separate from training data (no data leakage)

Typical split: 80% training, 20% held-out test data. You never train on test data.

FOR HIRING:

You need real hiring records: Candidates hired, how they performed, candidates rejected, etc. This is your ground truth.

COMPONENT 4: RUN THE EVALUATION

Three sub-steps:

a) Establish baseline: What's the performance of a simple approach?

Example baseline for hiring: A random classifier (coin flip). What % would it get right? 50%.

Better baseline: A simple rule (e.g., 'hire anyone with 5+ years experience'). What % correct?

Your system should beat baselines, not just be 'good' in absolute terms.

b) Evaluate your system: Run it on your test data, calculate metrics.

c) Compare: Your system's metrics vs baseline metrics.

Example: Random: 50% | Simple rule: 65% | Your AI: 82%

Your AI beats both baselines. Good sign.

COMPONENT 5: ANALYZE RESULTS

Don't just report numbers. Analyze what they mean.

Questions to ask:

- Is the improvement meaningful? (5% or 50%?)
- Is it statistically significant? (Could it be random chance?)
- Where does the system do well? (What types of candidates?)
- Where does it fail? (Where does it make mistakes?)
- Are there patterns in failures? (Bias, edge cases?)
- How does it compare to human performance? (Gold standard: expert hiring manager)

COMPONENT 6: MAKE DECISIONS

Based on analysis, what do you do?

Scenarios:

- System beats baseline by 20%, statistically significant, no bias detected → Deploy
- System beats baseline by 2%, not statistically significant → Don't deploy (not good enough)
- System beats baseline by 15% but biased against women → Don't deploy (fix bias first)
- System beats baseline by 10% but fails on edge cases → Deploy with caveats (humans handle edge cases)

REAL EXAMPLE: MEDICAL AI EVALUATION FRAMEWORK

Goal: 'This AI should match specialist diagnosis 93% of the time (meeting clinical needs) while flagging uncertain cases for human review'

Metrics: Accuracy (does it match specialist?), Sensitivity (catches actual positives?), Specificity (avoids false alarms?), Calibration (when it's 90% confident, is it right 90% of the time?)

Data: 500 cases from three hospitals (diverse data), labeled by multiple specialists (high-quality labels), split 80/20 train/test

Baseline: Specialist judgment from one doctor (82% accuracy with other specialists)

Evaluation:

- Random classifier: 50%
- Simple rule-based system: 75%
- Your AI model: 91%
- Specialist average: 93%

Analysis:

- AI gets 91% vs specialist 93% → Close to human level (good)
- Where AI fails: mostly on rare conditions (expected)
- AI is unbiased across demographics (checked)
- AI is well-calibrated (when 90% confident, it's right 89% of the time)

Decision: Deploy, but require human review on cases where AI confidence < 70%. This way, you get AI speed for high-confidence cases, human expertise for uncertain cases.

THE FRAMEWORK IN ACTION:

This isn't abstract. It's concrete and repeatable. Every product company uses something like this before shipping.

The key is: you're not just checking if something works. You're systematically proving it works, understanding where it works well and where it struggles, and making informed decisions based on evidence.

DELIVERY TIPS

The six-component framework is the core of this section. Make sure students understand each part. • Real medical example shows this isn't theoretical. • Emphasize the decision-making aspect: evaluation leads to actions. • Components are sequential but not always linear: sometimes you discover your metrics don't make sense and need to go back and redefine them.

05. KEY METRICS FOR AI SYSTEMS

~8 minutes

WHAT'S ON SCREEN

Metrics overview • Classification: Accuracy, Precision, Recall, F1 • Ranking: NDCG, MRR • Generation: BLEU, ROUGE • Real examples of each

SPEAKING SCRIPT

Now let's talk about specific metrics. Because different types of AI problems need different metrics.

CLASSIFICATION METRICS (For tasks with clear right/wrong answers)

Example: Is this resume a 'strong match' or 'weak match'? Binary classification.

ACCURACY: Percentage correct overall

Formula: $(\text{Correct predictions}) / (\text{Total predictions})$

Example: Out of 100 hiring decisions, AI got 82 correct → Accuracy = 82%

Limitation: If your dataset is imbalanced (90% strong matches, 10% weak matches), a dumb classifier that says 'all strong matches' would get 90% accuracy. Misleading.

PRECISION: Of predictions of 'strong match', how many actually are?

Formula: $(\text{True positives}) / (\text{True positives} + \text{False positives})$

Example: AI predicted 'strong match' 60 times. Of those, 48 were actually strong matches.

Precision = $48/60 = 80\%$

Meaning: When AI says 'strong match', you can be 80% confident it's correct

Use when: You want to avoid false positives (you don't want to wrongly hire bad candidates)

RECALL: Of actual strong matches, how many did AI find?

Formula: $(\text{True positives}) / (\text{True positives} + \text{False negatives})$

Example: There were actually 100 strong matches in the data. AI found 80 of them.

Recall = $80/100 = 80\%$

Meaning: AI found 80% of the strong candidates in the applicant pool

Use when: You want to avoid false negatives (you don't want to miss good candidates)

PRECISION vs RECALL TRADEOFF:

Imagine you're a hiring manager. You can adjust the AI to be more cautious (higher precision, lower recall) or more permissive (lower precision, higher recall).

High precision, low recall: AI only recommends the candidates it's most confident about. You don't miss false hires, but you might miss some good candidates.

Low precision, high recall: AI recommends many candidates. You find most good candidates, but also get some false recommendations you have to filter through.

Which is right? Depends on your problem. If hiring is expensive and a bad hire is very costly, you want high precision. If you're worried about missing top talent, you want high recall. Many systems aim for a balance.

F1 SCORE: Harmonic mean of Precision and Recall

Formula: $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

Use when: You want a single metric that balances both precision and recall

Example: Precision 80%, Recall 75% → F1 = 77.3%

ROC CURVE AND AUC:

Plots precision vs recall at different confidence thresholds. Gives you a visualization of the tradeoff.

AUC (Area Under Curve): Single number (0-1) representing overall performance across all thresholds. 0.5 = random guess, 1.0 = perfect.

Use when: You want to compare models across different operating points, or you want a single summary metric.

CONFUSION MATRIX: The full picture

Predicted Positive | Predicted Negative

Actual Positive: True Pos (TP) | False Neg (FN)

Actual Negative: False Pos (FP) | True Neg (TN)

Shows all four outcomes. From here, you can calculate precision, recall, specificity, etc.

RANKING METRICS (For tasks where order matters)

Example: Given a job description, rank candidates from best to worst match. The ranking matters, not just binary classification.

NDCG (Normalized Discounted Cumulative Gain):

Measures how good is your ranking, with emphasis on getting the best candidates at the top.

Range: 0-1 (higher is better)

Why 'discounted': Candidates at position 1 matter more than position 10

Example: If the two best candidates are in positions 1 and 2, high NDCG. If they're in positions 9 and 10, low NDCG.

Use when: Ranking quality matters more than binary accuracy

MRR (Mean Reciprocal Rank):

Measures position of the first relevant result

Example: If best candidate is in position 3, $MRR = 1/3 = 0.33$

If best candidate is in position 1, $MRR = 1/1 = 1.0$

Use when: You care most about finding the best item quickly

GENERATION METRICS (For tasks where AI generates text)

Example: AI generates a cover letter based on resume. Did it generate something good?

BLEU (Bilingual Evaluation Understudy):

Measures n-gram overlap between generated and reference text

Range: 0-1 (higher is better)

Example: Generated text shares 80% of 2-grams with reference $\rightarrow BLEU \approx 0.8$

Limitation: Doesn't understand meaning, only word overlap

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):

Similar to BLEU but emphasizes recall (capturing all important words from reference)

Use when: You're evaluating summaries or paraphrasing

PERPLEXITY:

Measures how surprised the model is by the test data (lower is better)

Use when: You want to measure language quality of generation

SEMANTIC SIMILARITY (Using embeddings):

Convert both generated and reference to embeddings, measure cosine similarity

More meaningful than BLEU because it captures semantic similarity, not just word overlap

Example: 'The quick fox' and 'The fast fox' are semantically similar even if different words

HUMAN EVALUATION METRICS:

Sometimes you need humans to score quality.

Likert Scale: Rate on 1-5 (1=terrible, 5=excellent)

Pairwise comparison: Which is better, A or B?

Ranking: Rank these candidates from best to worst

DOMAIN-SPECIFIC METRICS:

For hiring:

- Hire success rate: Of candidates the AI recommended, what % succeeded in the role?
- Diversity: What % of recommendations are from underrepresented groups?
- Time to hire: Did AI speed up or slow down hiring process?

For medical AI:

- Sensitivity: Catches actual positives?
- Specificity: Avoids false alarms?
- Calibration: Is AI confidence calibrated to accuracy?
- Differential accuracy: Does AI perform equally across demographics?

For customer service:

- Resolution rate: First-contact resolution?
- Customer satisfaction: CSAT score?
- Escalation rate: How often does AI escalate to human?

CHOOSING METRICS FOR YOUR PROBLEM:

Ask:

1. What's the goal? (What does success look like?)
2. What would be bad failure modes? (What mistakes are unacceptable?)
3. What can you measure? (What data do you have?)
4. What's the domain-specific requirement? (What matters in your field?)

Usually you pick 3-5 metrics. Too few misses important aspects. Too many is confusing.

EXAMPLE: HIRING AI METRICS

Primary metric: Recall on top-quartile candidates (don't miss good people)

Secondary metric: Precision on recommendations (don't overwhelm with bad recommendations)

Tertiary metric: Diversity (recommendations reflect diverse talent pool)

Safety metric: Bias check (no demographic disparities in false negative rates)

If your system excels on primary but fails on diversity, it's not ready to deploy.

DELIVERY TIPS

Metrics are the heart of evaluation. Take time here. • Show the classification metrics progression: accuracy → precision/recall → F1. • The precision-recall tradeoff is critical: most students get this after seeing it in context. • Real hiring example (with specific percentages) makes it concrete. • Domain-specific metrics show this isn't one-size-fits-all. • The final example ties everything to their project.

06. HUMAN EVALUATION

~8 minutes

WHAT'S ON SCREEN

Human evaluation process • Expert review, Rating scales, Annotation guidelines • Inter-rater agreement • When to use humans vs automatic

SPEAKING SCRIPT

Sometimes the best judge is a human. Especially when tasks are nuanced or where automatic metrics fail.

WHEN TO USE HUMAN EVALUATION:

Use when:

- Automatic metrics don't exist or are unreliable
- Task requires subjective judgment (is this response helpful? Is this bias present?)
- You need to validate automatic metrics against human judgment
- Cost of errors is high (medical, legal, financial decisions)
- You need to catch edge cases automatic tests miss

Examples:

- Rating quality of generated text: Is this cover letter good? (Subjective)
- Detecting bias: Does this hiring decision show demographic bias? (Requires expert judgment)
- Evaluating recommendation quality: Would you actually use this candidate recommendation? (Domain expertise needed)

THE HUMAN EVALUATION PROCESS:

STEP 1: Define evaluation criteria

What are you asking humans to evaluate? Be specific.

Bad: 'Is this resume evaluation good?'

Good: 'Rate the hiring recommendation on: (1) Does it correctly identify strong candidates? (2) Is it considering relevant experience? (3) Is the reasoning clear? (4) Could there be bias?'

STEP 2: Create annotation guidelines

Give evaluators clear instructions on how to evaluate.

Example for hiring:

'Rate resume evaluation on accuracy: 5 = Recommendation perfectly matches expected decision by senior hiring manager, 4 = Minor errors, 3 = Some errors but reasoning sound, 2 = Major errors, 1 = Completely wrong'

Why guidelines matter: Without them, different evaluators rate differently. Guidelines ensure consistency.

STEP 3: Select evaluators

Who should evaluate?

- Domain experts (best quality, expensive)
- Trained crowd workers (cheaper, variable quality)
- Internal team members (fast, biased)
- Mix of above (balanced approach)

Typical approach: 2-3 independent evaluators per item. If they disagree, discuss and resolve.

STEP 4: Evaluate sample

Don't evaluate entire dataset. Sample is fine.

- Small evaluation: 50-100 items

- Medium: 200-500 items
- Large (if budget allows): 1000+

Sample should be representative (mix of easy and hard cases, diverse examples)

STEP 5: Measure agreement

Do evaluators agree? If not, guidelines weren't clear or task is ambiguous.

Cohen's Kappa: Measures inter-rater agreement

Range: -1 to 1

0.8+ = Strong agreement (good)

0.6-0.8 = Moderate (acceptable)

< 0.6 = Weak (rethink your evaluation)

If kappa is low: Revise guidelines, re-evaluate

STEP 6: Aggregate scores

If 3 evaluators rate each item, how do you combine?

- Average: Treat scores as numbers, average them
- Majority: If 2 out of 3 say 'good', it's good
- Expert: Trust the most experienced evaluator if others disagree

STEP 7: Analyze results

What percentage of items got each rating?

Do ratings correlate with automatic metrics? (Validation)

Where do evaluators disagree? (Ambiguous cases)

REAL EXAMPLE: EVALUATING CUSTOMER SERVICE BOT

Task: Evaluate how well AI customer service bot responds to complaints

Criteria: (1) Did bot understand the issue? (2) Was response helpful? (3) Was tone appropriate? (4) Would customer be satisfied?

Scale: 1-5 (1=terrible, 5=excellent)

Guidelines:

'Understanding (1-5): 5 = Bot clearly identified core issue, 4 = Bot understood most of issue, 3 = Bot understood part of issue, 2 = Bot misunderstood, 1 = Bot completely missed issue'

Evaluators: 3 customer service managers (domain experts)

Sample: 100 customer complaints and bot responses

Results:

- Agreement on understanding: Kappa 0.82 (strong)
- Average understanding score: 3.7/5
- Average helpfulness score: 3.4/5
- 65% would result in satisfied customer, 35% would need escalation

Analysis: Bot is pretty good but misses understanding on some issues. Escalation rate of 35% is high—you'd want to improve to 10-15%.

Decision: Deploy with mandatory escalation path. Humans handle the 35% of cases bot doesn't handle well.

COST OF HUMAN EVALUATION:

Evaluation cost depends on:

- Complexity of task (simple tasks = cheaper) ■r/>- Expert level needed (experts = expensive)
- Scale (100 items vs 1000 items)
- Turn-around time (fast = expensive)

Typical costs:

- Crowd workers: \$1-5 per item
- Internal team: \$10-20 per item (salary cost)
- Expert consultants: \$50+ per item

Example: Evaluating 200 items with 3 raters at \$3/item = \$1,800

Is it worth it? If the system affects 10,000 users and costs \$100K to deploy, \$1,800 evaluation is minimal cost.

HUMAN EVALUATION + AUTOMATIC METRICS (Best practice)

Step 1: Human evaluation on sample (detailed, expensive)

Step 2: Develop automatic metric that correlates with human ratings

Step 3: Use automatic metric for ongoing monitoring (cheap, scalable)

Step 4: Re-sample with human evaluation occasionally to validate automatic metric still works

This combines best of both: Human judgment for development, automatic metrics for deployment.

DELIVERY TIPS

Human evaluation feels subjective but is actually highly structured. Show this. • The inter-rater agreement concept is important—shows you're measuring reliably. • Real customer service example is concrete and relatable. • Cost analysis shows this is practical: small cost relative to system value. • Final section (combining human + automatic) shows professional approach.

07. AUTOMATIC METRICS

~8 minutes

WHAT'S ON SCREEN

Automatic metrics advantages • Implementation examples • When they work well and when they fail • Validation against human judgment

SPEAKING SCRIPT

Automatic metrics are scalable, objective, and repeatable. But they can be misleading. Let me explain the tradeoffs.

ADVANTAGES OF AUTOMATIC METRICS:

Scale: Can evaluate on millions of examples

Example: Evaluating search ranking on 1 billion queries daily. Humans can't do this.

Speed: Instant results

Example: A/B test results available in seconds

Objectivity: No human bias or inconsistency

Example: Metric calculated same way every time

Cost: Very cheap (just computation)

Repeatability: Can run same evaluation any time

DISADVANTAGES OF AUTOMATIC METRICS:

They might not capture what matters

Example: BLEU score (word overlap) doesn't measure if generated text is actually helpful

They can be gamed

Example: System optimizes for BLEU and produces weird but high-BLEU text

They miss edge cases

Example: Accuracy metric doesn't show rare cases where system fails badly

They assume ground truth is correct

Example: If labeled data has errors, metric is measuring noise

IMPLEMENTING AUTOMATIC METRICS:

EXAMPLE 1: RANKING METRIC

Task: Given job description, rank candidates from best to worst match

Ground truth: Expert ranking of candidates for this role

Automatic metric: Rank candidates using your system, compare to expert ranking

Calculation: NDCG@10 (how good is your ranking in top 10?)

Simple implementation (pseudocode):

```
function evaluate_ranking(system_ranking, expert_ranking):  
  dcg = calculate_dcg(system_ranking) // idcg = calculate_dcg(expert_ranking) // ideal ranking  
  idcg // return ndcg // 0 to 1
```

EXAMPLE 2: CLASSIFICATION METRIC

Task: Predict 'strong match' or 'weak match' for candidate

Ground truth: Actual hiring outcome (hired and succeeded = strong, rejected or failed = weak)

Automatic metrics: Accuracy, Precision, Recall, F1

Implementation (pseudocode):

```
function evaluate_classification(predictions, ground_truth):
tp = count(pred='strong' AND true='strong')
fp = count(pred='strong' AND true='weak')
fn = count(pred='weak' AND true='strong')
tn = count(pred='weak' AND true='weak')
accuracy = (tp + tn) / (tp + fp + fn + tn)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * (precision * recall) / (precision + recall)
return {accuracy, precision, recall, f1}
```

EXAMPLE 3: SEMANTIC SIMILARITY METRIC

Task: AI generates a job description for a role. How similar is it to a good reference?

Ground truth: Hand-written reference job description

Automatic metric: Semantic similarity using embeddings

Implementation (high-level):

```
function evaluate_generation(generated_text, reference_text):
gen_embedding = embed(generated_text)
ref_embedding = embed(reference_text)
similarity = cosine_similarity(gen_embedding, ref_embedding)
return similarity // 0 to 1
```

Better than BLEU because it measures meaning, not just word overlap

WHEN AUTOMATIC METRICS WORK WELL:

Tasks with clear right/wrong answers

Example: Classification (matched or not), ranking (order matters)

Tasks with objective ground truth

Example: Medical diagnosis (tests show you were right or wrong)

Tasks where you have lots of labeled data

Example: You can validate metric against human judgment

WHEN AUTOMATIC METRICS FAIL:

Tasks with subjective answers

Example: 'Is this job description good?' - Multiple good answers possible

Tasks where diversity matters

Example: Generated cover letter. Multiple different good versions exist. Metric penalizes diversity.

Tasks with rare edge cases

Example: 99% accuracy but 1% of cases where system completely fails dangerously

Tasks requiring common sense

Example: Detecting harmful content. Need human judgment for context.

VALIDATING AUTOMATIC METRICS:

Don't assume your automatic metric is right. Validate it.

Process:

1. Define automatic metric
2. Evaluate 100+ examples with automatic metric
3. Have humans evaluate same 100 examples
4. Compare: Do automatic metric and human judgment agree?
5. If correlation is high ($r > 0.8$), metric is validated
6. If correlation is low, metric needs work

REAL EXAMPLE: VALIDATING HIRING METRIC

Automatic metric: 'AI recommendation accuracy' (% matches expert recommendation)

Validation:

1. Run metric on 100 resumes: AI got 82% accuracy
2. Have 3 expert hiring managers evaluate same 100: Managers think AI quality is actually 75% (lower)

Analysis: AI looks good on automatic metric but humans rate it lower

Reason: Automatic metric is comparing to one expert's recommendation. But different experts disagree. Humans judge on 'could this work' not 'does this match expert'

Learning: Automatic metric not good proxy for actual quality. Need more nuanced metric or more human judgment.

METRIC GAMING (Important concern):

If you optimize solely for a metric, system might achieve high metric in weird ways.

Example: Customer service AI optimizes for 'conversation length' (think shorter is better)

- Good optimization: Resolves issues quickly
- Bad optimization: Abruptly ends conversation even when customer isn't satisfied

Solution: Multiple metrics. Can't game all of them.

Example: Track 'resolution rate' AND 'customer satisfaction' AND 'conversation length'

System can't just minimize one.

COMBINING MULTIPLE METRICS:

No single metric is perfect. Use multiple.

Typical combination:

- Primary metric (most important for your goal)
- Secondary metrics (important but secondary)
- Guardrail metrics (things that shouldn't get worse)
- Diversity metrics (system should work across different groups)

Example for hiring:

- Primary: Recall on successful candidates (find good people)
- Secondary: Precision (don't overwhelm with bad recommendations)
- Guardrail: Bias (no demographic disparities)
- Diversity: Recommendation rate by demographic (should be similar across groups)

A system passes if it improves on primary and secondary while maintaining guardrails and diversity.

DELIVERY TIPS

Automatic metrics are tempting because they're easy, but they're also easy to get wrong. Balance is key. • The pseudocode examples make implementation concrete. • Metric gaming is a real problem in ML—share this honestly. • The validation process (automatic vs human agreement) is essential. • Real hiring example shows practical tradeoffs.

08. A/B TESTING & STATISTICAL SIGNIFICANCE

~8 minutes

WHAT'S ON SCREEN

A/B test framework • Treatment vs Control • Statistical significance • P-values and confidence intervals • Sample size calculation

SPEAKING SCRIPT

This is critical. You've measured that your system is better. But is the improvement real or just random chance?

THE PROBLEM:

You run your evaluation: Old system 80% accuracy, new system 82% accuracy. 2% improvement. Should you ship the new system?

Not necessarily. That 2% could be random variation.

Think about it: If you flip a coin 10 times, you might get 6 heads (60%). Flip again, you might get 4 heads (40%). Same coin, different results. Random variation.

With evaluation, same thing happens. Test on dataset A: system might perform 80%. Test on dataset B: might perform 82%. Not because system improved. Just random variation.

Statistical significance testing tells you: Is the difference real or random chance?

A/B TESTING FRAMEWORK:

STEP 1: Define hypothesis

Null hypothesis: 'Old and new system perform equally'

Alternative hypothesis: 'New system performs better'

STEP 2: Decide on test metrics

What will you measure? Accuracy? Revenue per user? Customer satisfaction?

Pick before the test. Don't pick based on which makes new system look good.

STEP 3: Split traffic (or data)

Control group: Gets old system

Treatment group: Gets new system

Important: Random split. Don't cherry-pick data for treatment group.

For hiring: If you have 1000 hiring decisions, randomly assign 500 to use old system, 500 to use new system

STEP 4: Collect data

Run both systems, collect results

STEP 5: Statistical test

Run a statistical test to determine if difference is significant

Common tests:

- T-test: For continuous metrics (average salary, average rating)
- Chi-square test: For categorical metrics (pass/fail, yes/no)
- Proportions test: For rates (% of users who convert)

STEP 6: Interpret results

Get a p-value (probability that null hypothesis is true)

IF p-value < 0.05: Likely real difference (statistically significant)

IF p-value $\geq$ 0.05: Likely random chance (not significant)

REAL EXAMPLE: HIRING SYSTEM A/B TEST

Hypothesis: New hiring recommendation system improves hire success rate

Metrics: Success rate (% of recommended candidates who succeed in role)

Data: 1000 hiring decisions from past 6 months

- 500 random decisions: Use old system (baseline)

- 500 random decisions: Use new system (treatment)

Results:

Old system: 72/500 candidates succeeded = 14.4% success rate

New system: 78/500 candidates succeeded = 15.6% success rate

Difference: 1.2 percentage points

Statistical test (proportions test):

p-value = 0.23

Interpretation: p=0.23 means there's 23% chance this difference is random. That's high. We want $p < 0.05$ (5% chance of random).

Decision: Difference not statistically significant. Don't deploy new system based on this data.

ANOTHER EXAMPLE: Same scenario, bigger sample

Test on 10,000 hiring decisions instead of 1,000

- Old: 1,420 successes / 5,000 = 14.2%

- New: 1,560 successes / 5,000 = 15.6%

Same 1.4 percentage point difference. But with bigger sample:

p-value = 0.002

Interpretation: p=0.002 means only 0.2% chance this is random. Very likely real difference.

Decision: Difference IS statistically significant. Deploy new system.

This shows: Sample size matters. With small sample, variation dominates. With large sample, real differences emerge from noise.

STATISTICAL SIGNIFICANCE vs PRACTICAL SIGNIFICANCE:

Just because something is statistically significant doesn't mean it matters.

Example: With 100,000 data points, even 0.1% difference might be statistically significant ($p < 0.05$)

But 0.1% improvement might not be worth deployment costs.

So ask both questions:

1. Is it statistically significant? (Is it real?)

2. Is it practically significant? (Does it matter?)

CONFIDENCE INTERVALS:

Instead of just p-values, also report confidence intervals.

Example: 'New system success rate is 15.6% with 95% confidence interval [14.2%, 17.1%]'

Meaning: Based on data, true success rate is likely between 14.2% and 17.1%. We're 95% confident about this range.

Why useful: Shows uncertainty in your estimate. Small interval = confident. Large interval = uncertain.

COMMON MISTAKES:

MISTAKE 1: P-hacking

You run multiple tests, report the one that shows significance, ignore the rest.

Example: Test 100 metrics, 5 show $p < 0.05$ by random chance. Report only those 5.

By random chance, 5% of 100 tests will show false significance. Don't do this.

Protection: Decide on metrics before test. Report all metrics.

MISTAKE 2: Stopping early

You're running experiment. Midway through, results look good. Stop and ship.

Problem: Early stopping increases false positive rate.

Protection: Decide on sample size before running test. Stick to it.

MISTAKE 3: Ignoring regression

New system is better on primary metric but worse on secondary metrics.

Example: 15% improvement in accuracy, but 30% more false positives.

Protection: Report on multiple metrics. A system must pass all, not just one.

SAMPLE SIZE CALCULATION:

How many samples do you need?

Simple rule of thumb: Need 385 samples per group to detect 5% relative improvement with 80% power at $p=0.05$

More samples = more confidence in results

But more samples = more time and cost

For hiring: If base rate is 14%, want to detect 15% (1 point improvement):

Need ~1,500 samples per group (3,000 total)

Takes ~1-2 months to collect

SEQUENTIAL TESTING:

Don't wait for all samples. Test continuously.

Example: Run test on 500 samples, analyze. Then 1000 samples, analyze. Etc.

Stop when statistically significant or when you've hit max budget.

Advantage: Can deploy faster if results are good

Disadvantage: More complex analysis needed (sequential testing adjusts p-value threshold)

FOR YOUR HIRING DOMAIN:

A/B test example:

- Hypothesis: Fine-tuned hiring AI beats base model
- Metrics: Success rate (hired, succeeded), Bias (demographic parity in hiring)
- Sample size: 1,000 hiring decisions (500 per group)
- Test duration: 2-3 months
- Significance level: $p = 0.05$
- Pass criteria: (1) New system has higher success rate, $p < 0.05$, (2) No demographic bias increase

DELIVERY TIPS

A/B testing and statistical significance are essential and often done wrong. Take time here. • The two hiring examples (first nonsignificant, second significant) show how sample size matters. • Common mistakes section prevents real errors.

- The distinction between statistical and practical significance is important.
- Sample size calculation shows this is practical planning, not abstract theory.

09. BUILDING BENCHMARK DATASETS

~5 minutes

WHAT'S ON SCREEN

Benchmark characteristics • Diverse data • Labeled correctly • Size sufficient • Public benchmarks vs proprietary • Quality over quantity

SPEAKING SCRIPT

What do you test on? That's crucial. Because bad test data leads to bad conclusions.

CHARACTERISTICS OF GOOD BENCHMARKS:

REPRESENTATIVE:

Data should be similar to real-world usage

Example for hiring: Your test data should include: diverse roles (engineer, salesman, manager), diverse backgrounds (new grad, experienced, career changers), diverse locations, diverse demographics

If test data only includes tech roles, your evaluation doesn't tell you how system performs on non-tech roles

DIVERSE:

Not just representative—diverse within representation

Example: Don't have all software engineers be from same background. Include some from bootcamps, some from colleges, some self-taught

Because system might have biases you don't see without diversity

LABELED CORRECTLY:

Ground truth must be accurate

Example for hiring: Ground truth is 'did this person succeed in the role'

If you hire someone and they succeed, that's positive. But what defines 'succeed'? 6-month retention? Performance rating > 3/5? Be explicit.

Bad labeling: 'Hire if they seem good'—too vague

Good labeling: 'Hire if stayed > 6 months AND performance >= 3.5/5'

SUFFICIENT SIZE:

Big enough to draw conclusions

- Small benchmark (50 examples): High variance, unreliable

- Medium benchmark (500 examples): Reasonable

- Large benchmark (5000+): Good

Size depends on: task complexity, how many metrics you're measuring, how rare negative cases are

NO LEAKAGE:

Test data must be completely separate from training data

Example: If you fine-tuned your model on some hiring data, don't test on that same data. Use different data collected separately.

Data leakage makes systems look better than they are

REPRODUCIBLE:

Future researchers/competitors should be able to use same benchmark and compare

This is why many companies publish their benchmarks (MNIST, ImageNet, SQuAD for NLP)

PUBLIC VS PROPRIETARY BENCHMARKS:

PUBLIC BENCHMARKS (Published datasets):

Examples: ImageNet (vision), MNIST (handwriting), SQuAD (reading comprehension), Common Voice (speech)

Advantages: Standard, reproducible, competitive (can compare to others)

Disadvantages: Might not match your domain. Might be outdated.

Use case: Development, research, proving general capability

PROPRIETARY BENCHMARKS (Your data):

Examples: Your company's resumes, your customers' questions, your platform's interactions

Advantages: Realistic, matches your domain exactly, captures your specific challenges

Disadvantages: Can't compare to others, harder to share results publicly

Use case: Production evaluation, domain-specific optimization

BEST PRACTICE: Use both

Phase 1 (Development): Use public benchmarks to develop system. Compare to others.

Phase 2 (Optimization): Use proprietary benchmarks to optimize for your domain.

Phase 3 (Validation): Validate on both public (general capability) and proprietary (domain capability).

CREATING YOUR OWN BENCHMARK:

1. COLLECT DATA

What's your data source? Your own logs? Historical data? New data collection?

For hiring: Could use past 6 months of hiring records

2. LABEL DATA

What's ground truth? Be explicit.

For hiring: Define success (e.g., 6+ months employment AND performance $\geq 3.5/5$)

Label data accordingly

3. QUALITY CHECK

Sample check: Review 10% of labels manually

Are labels correct? Consistent?

If error rate $> 5\%$, re-label or clarify guidelines

4. ANALYZE DISTRIBUTION

What's the split of classes?

Example: If 90% positive (hired and succeeded) and 10% negative (rejected or failed), that's imbalanced

Imbalanced data: High accuracy baseline (always predict positive) gets 90% right without being useful

5. SPLIT INTO TRAIN/VALIDATION/TEST

Typical: 60% train, 20% validation, 20% test

Or: 70% train, 30% test

Keep stratified: If overall data is 90% positive, each split should be $\sim 90\%$ positive

6. VERSION AND DOCUMENT

Version your benchmark (V1, V2, etc.)

Document: Source, size, class distribution, how labels were created, any known issues

If you update benchmark, others should be able to track what changed

BENCHMARK SIZE FOR YOUR HIRING DOMAIN:

Minimum viable: 500 examples (250 hired and succeeded, 250 hired and failed or rejected)

Good: 1,000 examples

Excellent: 5,000+ examples

Factors affecting size:

- How rare is the negative case? If 95% of hires succeed, need bigger benchmark to evaluate on enough failures
- How many metrics? More metrics need bigger benchmark
- How much variation? If hiring very consistent, smaller benchmark okay. If highly variable, need larger.

USING PUBLIC BENCHMARKS FOR COMPARISON:

If available, use public hiring benchmarks to compare your system

Example: Academic dataset of 1,000 hiring decisions with labels

Compare your system to existing systems on same benchmark

Shows relative performance

BENCHMARK DRIFT:

Over time, data distribution changes

Example: In hiring, once you deploy your AI, data changes because you're now making different hiring decisions

Solution: Occasionally collect new data for re-evaluation

Check: Is your benchmark still representative? Do you need to add new types of data?

Re-benchmark every 6-12 months

DELIVERY TIPS

Benchmark quality is foundational. Everything else flows from good data. • The representative vs diverse distinction is important: both matter. • Imbalanced data is a real issue that catches many people. • Public vs proprietary comparison shows strategy: start with public, optimize on proprietary. • Benchmark versioning is professional practice. • The specificity on hiring domain makes it concrete.

10. REAL-WORLD EVALUATION EXAMPLES

~8 minutes

WHAT'S ON SCREEN

5 Company examples • Google Search, OpenAI models, Medical diagnostic AI, Content moderation, Hiring platforms • Full evaluation process for each • Metrics, benchmarks, results

SPEAKING SCRIPT

Let me show you how real companies do evaluation. This is what production evaluation looks like.

EXAMPLE 1: GOOGLE SEARCH RANKING

Problem: Google needs to evaluate if a new ranking algorithm is better than current one

Metrics:

- Relevance: Do top results match search intent? (Human raters score 1-5)
- Click-through rate: What % of users click on result? (Automatic)
- Query reformulation: Do users search again immediately? (Automatic, low is good)
- Dwell time: How long do users stay on clicked result? (Automatic, higher is better)
- Search satisfaction: Post-search survey (Human)

Benchmark: ~150,000 sample queries

- Diverse queries (different languages, intent types, difficulty levels)
- Queries sampled randomly to avoid bias

Evaluation process:

1. Sample 10,000 queries
2. Run both algorithms on sampled queries
3. Have 100+ human raters score results (Relevance metric)
4. Measure automatic metrics on full 150,000
5. A/B test on 1% of search traffic for 1 week
6. Statistical analysis

Decision criteria:

- Must improve on primary metric (relevance)
- Must not worsen on secondary metrics (CTR, dwell time)
- Must pass statistical significance test ($p < 0.001$ given scale)
- Must not show demographic biases

If passes all: Full rollout

If ambiguous: More testing

If fails: Back to development

Real timeline: 2-4 weeks from algorithm to decision

EXAMPLE 2: OPENAI MODEL EVALUATION

Problem: OpenAI developed GPT-4. Need to verify it's actually better than GPT-3.5 before release

Evaluation approach:

1. Academic benchmarks: MMLU (45K multiple choice questions across diverse knowledge), GSM8K (math), MATH (harder math)
2. Professional exams: Bar exam, Medical licensing exam, SAT, GRE (simulate real-world usage)
3. Adversarial prompts: Try to break the model, look for failure modes
4. Safety evaluation: Can model produce harmful content? Is it aligned with values?
5. Human comparative evaluation: Side-by-side tests, which output is better?

Benchmarks:

- Academic: 50K+ diverse questions

- Professional: 10K+ questions from actual exams
- Safety: Custom 1000+ adversarial prompts

Results reported:

GPT-3.5 vs GPT-4 on public benchmarks

- MMLU: 86% vs 92% (+6 points)
- GSM8K: 80% vs 92% (+12 points)
- Bar exam: 90th percentile vs 88th percentile (GPT-4 better)
- Adversarial: GPT-3.5 fails on 30%, GPT-4 fails on 15% (safer)

But NOT just raw numbers. They also report:

- Areas where GPT-4 still fails (math reasoning, creative tasks)
- Limitations and risks
- Appropriate use cases

Decision: GPT-4 is sufficiently better on both capability and safety. Release with documentation about limitations.

Key insight: Transparent about what works and what doesn't. Don't just report wins.

EXAMPLE 3: MEDICAL DIAGNOSTIC AI

Problem: Hospital system deployed AI to help radiologists detect cancer

Evaluation (clinical trial format):

1. Gold standard: Biopsy (definitive diagnosis)
2. Benchmark: 500 patient images, verified by pathology
3. Radiologist evaluation: 5 experienced radiologists independently evaluate same images
4. AI evaluation: AI evaluates same images

Metrics:

- Sensitivity (recall): % of actual cancers detected
- Specificity: % of actual non-cancers correctly identified as non-cancer
- ROC curve: Sensitivity vs specificity across thresholds
- Differential performance: Does performance vary by demographic? (Age, gender, ethnicity)

Results:

Single radiologist: 85% sensitivity, 80% specificity

AI alone: 88% sensitivity, 78% specificity

AI + radiologist: 93% sensitivity, 82% specificity

Analysis: AI doesn't replace radiologist. But AI + radiologist is better than radiologist alone.

Demographic analysis:

- Performance similar across age groups (good)
- Small performance gap for different ethnicities (concerning, investigate)

Decision: Deploy as assistive tool (AI shows results to radiologist, who makes decision). Not autonomous.

EXAMPLE 4: CONTENT MODERATION AI

Problem: Platform has millions of posts daily. Need AI to flag policy violations efficiently

Metrics:

- Recall: % of actual violations AI flags (catch bad stuff)
- Precision: % of AI flags that are actually violations (don't waste human time)
- False negative rate: % of violations AI misses (safety metric)
- False positive rate: % of good content AI incorrectly flags (user experience)
- Bias: Are violation rates similar across demographics?

Benchmark: 10,000 posts manually reviewed by human moderators

- Representative mix of content

- Multiple reviewers per post for accuracy
- Inter-rater agreement > 0.8 before using as ground truth

Results by violation type:

- Hate speech: Recall 92%, Precision 85%
- Violence: Recall 88%, Precision 92%
- Spam: Recall 76%, Precision 95%
- Misinformation: Recall 45%, Precision 70% (hard, lots of nuance)

Overall policy: Misinformation too hard for AI alone. Require human judgment.

Deploy AI on: hate speech, violence, spam

Human review for: misinformation, context-dependent violations

EXAMPLE 5: HIRING PLATFORM AI

Problem: Platform helps companies screen resumes. Need to evaluate recommendation quality

Evaluation process:

1. Collect: 5,000 recent hiring decisions from platform users
2. Ground truth: Whether recommended candidates were actually hired and succeeded (6+ months, positive feedback)
3. Metrics: Recall (find good candidates), Precision (don't waste recruiter time)
4. Benchmark diversity: Different industries, different company sizes, different roles

Results by industry:

- Tech: Recall 82%, Precision 79% (good)
- Finance: Recall 68%, Precision 85% (recall lower, harder domain)
- Sales: Recall 75%, Precision 70% (mixed)

Demographic analysis (important for hiring):

- Recommendation rate for male: 45%
 - Recommendation rate for female: 44%
 - Recommendation rate for non-binary: 43%
- (Similar across groups = good, no demographic bias)

Success rate by demographic:

- Male recommended candidates: 78% succeeded
 - Female recommended candidates: 79% succeeded
 - Non-binary recommended candidates: 76% succeeded
- (Similar success rates = fair, no quality bias)

Decision: Deploy with monitoring for demographic parity

Commit to: Monthly re-evaluation of demographic metrics

Alert if any demographic shows < 75% recommendation rate or > 2% differential success rate

PATTERN ACROSS ALL FIVE:

1. Multiple metrics: No single metric tells full story
2. Domain expertise: Each evaluation designed for specific domain
3. Benchmarks matter: Used representative, diverse, high-quality data
4. Human judgment: Automatic metrics supplemented with human raters
5. Demographic analysis: Checked for bias (ethical requirement)
6. Transparent about limitations: Reported what works and what doesn't
7. Decision process: Clear criteria for pass/fail/deploy with caveats
8. Ongoing monitoring: Evaluation doesn't end at deployment

DELIVERY TIPS

Five real companies across different domains show this is universal practice. • Each example shows full end-to-end evaluation process. • Emphasis on transparency (reporting limitations) is important. • Demographic parity/fairness evaluation is now standard. • The progression from development (benchmarks) to deployment (monitoring) shows full lifecycle. • Students should see: evaluation is sophisticated, systematic, ongoing.

11. CONTINUOUS MONITORING

~5 minutes

WHAT'S ON SCREEN

Post-deployment monitoring • Metrics dashboards • Performance drift detection • Retraining triggers • Feedback loops

SPEAKING SCRIPT

Evaluation doesn't end at deployment. It continues. Because systems degrade over time.

WHY SYSTEMS DEGRADE:

Data drift: Real-world data changes over time

Example for hiring: Labor market changes. New role types emerge. Candidate demographics shift. Your model trained on 2022 data might not work well on 2024 candidates.

Concept drift: The problem itself changes

Example: What makes a 'successful hire' might change. Company grows, values change. Seniority level needed changes. Model trained to identify one type of success might miss new type.

Model degradation: Performance slowly decreases

Not because system changed. But because world changed.

User adaptation: Users learn to game the system

Example: Candidates figure out what hiring AI looks for, optimize resumes accordingly. AI sees artificially inflated qualifications.

CONTINUOUS MONITORING APPROACH:

STEP 1: Define monitoring metrics

Track same metrics as evaluation

Example for hiring: Track recall, precision, success rate, demographic parity monthly

Set alert thresholds

Example: Alert if recall drops below 75%, or if demographic parity diverges > 3%

STEP 2: Collect real-world data

For every decision AI makes, collect:

- Input (candidate info)
- AI decision (recommend or not)
- Outcome (hired or not)
- Result (if hired, did they succeed?)

This takes time. Outcome (whether hired) often immediate. Result (success) takes months.

STEP 3: Calculate metrics regularly

Monthly: Calculate on last month's data

Quarterly: Full re-evaluation on larger dataset

Yearly: Full benchmark re-run with new data

STEP 4: Detect drift

Plot metrics over time. Look for trends.

Example chart:

- Month 1: Recall 85%
- Month 2: Recall 84%

- Month 3: Recall 83%
- Month 4: Recall 82%
- Month 5: Recall 78% (Alert!)

Slow decline = drift. Sudden drop = bug.

STEP 5: Investigate and respond

If metrics degrade:

- Investigate cause: Data drift? Concept drift? System bug?
- Decide: Retrain? Roll back? Accept degradation?

Retraining trigger:

- Performance drops > 5% from baseline
- OR demographic parity changes > 3%
- OR new data distribution detected

STEP 6: Feedback loops

Use real-world outcomes to improve model

Example: You hired candidates AI recommended. Some succeeded, some didn't. Retrain on this new data so model learns from actual outcomes.

This is why data scientists prefer: logged decisions + outcomes > theoretical evaluation

PRACTICAL EXAMPLE: HIRING AI MONITORING

Month 1 (baseline):

- Recall: 85%
- Precision: 80%
- Success rate: 78%
- Male recommendation rate: 45%
- Female recommendation rate: 44%

Month 4:

- Recall: 82%
- Precision: 78%
- Success rate: 76%
- Male recommendation rate: 48%
- Female recommendation rate: 39%

Analysis: Recall slight decline (normal variance). But female recommendation rate dropped to 39%! Alert.

Investigation: Checked data. Found: More women received advanced degrees recently. Model trained pre-degree-inflation. Undervaluing women with new credentials.

Action: Retrain on 6 months of new data including recent degree patterns.

New results: Female recommendation rate back to 44%, performance maintained.

COST OF MONITORING:

Engineering cost: Dashboard, alerting system, logging infrastructure (~\$50-100K one-time)

Ongoing cost: Data storage, computation (~\$5-20K per month)

Team cost: 1 person part-time reviewing alerts and deciding on actions

ROI: Prevents big failures (bad deployment, lost reputation, regulatory issues)

AUTOMATED RETRAINING:

Some companies automate retraining:

1. Collect new data monthly
2. If new data sufficiently different from training data, trigger retraining automatically
3. Retrain model in offline process
4. Evaluate new model on hold-out test set
5. If new model passes criteria, deploy automatically
6. If not, alert humans

Advantage: Keeps system fresh without manual intervention

Risk: Automated deployment without human check can go wrong

BEST PRACTICE: Hybrid

- Automatic monitoring and alerting
- Automatic retraining offline
- Human review before production deployment

LONG-TERM EVALUATION:

Every year: Full re-evaluation on new benchmark

Every 2 years: Audit for bias and fairness

Every 3 years: Consider if approach still makes sense (might need fundamental change)

This is why companies have ML operations (MLOps) teams. Evaluation is continuous, not one-time.

DELIVERY TIPS

Monitoring is often overlooked but essential. Systems don't stay good forever. • Real drift example (women's education) shows why this matters. • The automated retraining discussion shows where the field is going. • Cost-benefit analysis justifies investment. • Emphasis on hybrid approach (automated monitoring, human decision) is realistic.

12. LAB EXERCISE OVERVIEW

~3 minutes

WHAT'S ON SCREEN

Lab title: 'Evaluate Your System Rigorously' • Steps: Create benchmark, Establish baseline, Evaluate system, Statistical test, Report results

SPEAKING SCRIPT

Alright, lab time. You're going to experience real evaluation. Not simplified, not theoretical. Real.

YOUR TASK: Create evaluation benchmark for your hiring domain. Establish baseline. Evaluate your system. Run statistical significance test. Report results rigorously.

STEP 1: CREATE BENCHMARK (2 min)

20-30 hiring scenarios with ground truth (strong match or weak match)

Use diverse scenarios

STEP 2: ESTABLISH BASELINE (1 min)

Simple approach: Random recommendations

Calculate baseline metrics

STEP 3: EVALUATE YOUR SYSTEM (2 min)

Run your engineered hiring system on benchmark

Calculate metrics: Accuracy, Precision, Recall, F1

STEP 4: STATISTICAL TEST (2 min)

Run statistical significance test comparing your system to baseline

Calculate p-value

STEP 5: ANALYZE AND REPORT (2 min)

Write results: Metrics, comparison, statistical significance

Discuss: Is improvement real or random?

STEP 6: REFLECTION (2 min)

What did you learn? What would you measure differently? How would you monitor in production?

DELIVERABLES:

1. Benchmark dataset (20-30 labeled examples)
2. Baseline metrics
3. Your system metrics
4. Statistical test results and p-value
5. Results report with interpretation
6. Reflection on evaluation design

DELIVERY TIPS

This is the capstone of Sessions 1-7. Students evaluate their own system. • Step-by-step process mirrors real-world evaluation. • Statistical testing is the key learning. • This makes evaluation concrete, not abstract.

13. KEY TAKEAWAYS FROM SESSION 7

~5 minutes

WHAT'S ON SCREEN

6 Core Principles • Why evaluation, Framework, Metrics, Human judgment, Statistical testing, Continuous monitoring

SPEAKING SCRIPT

Takeaway One: Evaluation Is Non-Negotiable

Without rigorous evaluation, you don't know if your system works. And shipping without knowing is risky.

Takeaway Two: Systematic Framework Beats Ad-Hoc Testing

Define goal → Choose metrics → Gather data → Establish baseline → Evaluate system → Analyze results → Decide

Takeaway Three: Multiple Metrics Required

No single metric tells full story. Accuracy alone can mislead. Use primary, secondary, guardrail, and diversity metrics.

Takeaway Four: Combine Human Judgment and Automatic Metrics

Automatic metrics are scalable. Human judgment is accurate. Use both. Neither alone is sufficient.

Takeaway Five: Statistical Significance Is Essential

Improvements must be statistically significant, not random chance. A/B test properly. Calculate p-values.

Takeaway Six: Monitoring Never Stops

Deployment is not the end. Systems degrade. Monitor continuously. Retrain as needed. Evaluation is ongoing.

DELIVERY TIPS

These six principles tie everything together. • Emphasis on systematic thinking and rigor. • Statistical significance is highlighted because it's often skipped. • Continuous monitoring shows maturity: evaluation is lifecycle, not project.

14. COMING UP NEXT

~2 minutes

WHAT'S ON SCREEN

Dark blue background • Title: 'Session 8: Scaling & Production' • Preview bullets • Closing remarks

SPEAKING SCRIPT

Session 8 is where your system goes from laboratory to reality. You've built it. You've evaluated it. Now: how do you scale it to serve millions of users?

Scaling isn't just infrastructure. It's about maintaining quality, handling edge cases, monitoring at scale, deploying safely.

You'll learn: How to architect for scale. How to deploy incrementally. How to handle failures. How real companies go from 1 user to 1 million users without losing quality.

Thank you all for today. You now know how to rigorously evaluate AI systems. This is the bridge between research and production. This is where systems become products. See you in Session 8.

DELIVERY TIPS

Session 8 natural progression: You've evaluated, now scale. • Emphasize the bridge between research (Sessions 1-7) and production (Sessions 8-12). • End with impact: 'This is where systems become products.'